

Software Architecture

91.1	Underlying Principles	91-1
	Definition • Rationale for Architecture	
91.2	Architecture Modeling	91-3
91.3	Architecture Design Methods	91-5
	Analyze Concerns • Analyze Domain • Architecture Design • Architecture Evaluation • Architecture Realization	
91.4	Architecture Evaluation.....	91-9
91.5	Research Issues.....	91-10
	Modeling Quality Concerns • Model-Driven Approach for Architecture Design	
91.6	Summary.....	91-14
	Key Terms.....	91-14
	References.....	91-15

Bedir Tekinerdogan
Bilkent University

91.1 Underlying Principles

The term *architecture* has been used for centuries to denote the art and science of designing buildings and structures. The term is applied in various different domains scope including building architecture, town planning, urban design, and landscape architecture. One of the earliest written works on the topic of architecture is *De Architectura* (On Architecture), which was published as Ten Books on Architecture, by the Roman architect Vitruvius in the early first century CE [10]. The work is a treatise on architecture and served as a guide for building projects. Currently, the work is one of the most important sources of knowledge of Roman building methods and the planning and design of both large and small structures, including buildings, aqueducts, baths, harbors, and machines. The concept of architecture has further evolved in the history through various traditions and regions.

The software engineering community has also adopted the term architecture to denote the gross-level design of software-intensive systems. The importance of high-level architectural structure was already acknowledged early in the history of software engineering. The first software programs were written for numerical calculations using programming languages that supported mathematical expressions and later algorithms and abstract data types. Programs written at that time served mainly one purpose and were relatively simple compared to the current large-scale diverse software systems. Over time, due to the increasing complexity and the increasing size of the applications, the global structure of the software system became an important issue.

Currently, it is not the algorithms and data structures that constitute the major design problems, but the construction of the systems from many components and the organization of the overall system [16]. Already in 1968, Dijkstra proposed the correct arrangement of the structure of software systems before simply programming [13]. He introduced the notion of layered structure in operating systems, in which related programs were grouped into separate layers, communicating with groups of programs in adjacent layers. Later, Parnas argued that the selected criteria for the decomposition of a system impact the structure

of the programs and several design principles must be followed to provide a good structure [27,28]. Within the software engineering community, there is now an increasing consensus that the architectural design of software systems is important to cope with the increased complexity and meet the stakeholder requirements properly. Without loss of generality, we can state that software architecture is recognized as a key determinant of project success. Without the right architecture, the project will likely fail.

91.1.1 Definition

In tandem with the increasing popularity of software architecture design, many definitions of architecture have been introduced over the last decade, though a consensus on a standard definition is still not established. We think that the reason why so many and various definitions on software architectures exist is because every author approaches a different perspective of the same concept of software architecture and likewise provides a definition from that perspective. Notwithstanding the numerous definitions, it appears that the prevailing definitions do not generally conflict with each other and commonly agree that software architecture represents the gross-level design of the software system consisting of components and relations among them.

The ISO/IEC 42010 (formerly IEEE 1471) Recommended Practice for Architectural Description of Software-Intensive Systems provides the following definition for software architecture [19]:

Architecture is the fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution.

Another recent definition is given in [8]:

The software architecture of a program or computing system is the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.

There are plenty of other definitions, each focusing on different aspects. Based on the definitions in the literature, we consider the following definition of software architecture that includes the important aspects:

Architecture represents the gross level structure of software intensive systems, which consists of software components and relationship among these components, including systemic design decisions and the rationale for these design decisions, to meet the set of concerns for the stakeholders.

The aforementioned definition has in fact three parts: (1) the structure and behavior of the system, (2) the design decisions that have led to the structure together with the rationale behind these design decisions, and (3) the stakeholder concerns that govern the design decisions and shape the architecture. The term component here is used as an abstraction of different interpretations and likewise may refer to sub-systems, processes, software modules, hardware components, or any element that has systemic behavior. In a similar sense, relations may refer to data flows, control flows, call relations, part of relations, etc.

91.1.2 Rationale for Architecture

Software architecture is an abstract representation that serves various purposes. We list the following:

- *Architecture helps to understand the system*

An important concern in software projects is the understanding of the system and its properties. Software architecture is a high-level abstraction of the system that abstracts away from the irrelevant details and as such can facilitate the understandability.

- *Architecture supports communication among stakeholders*
Software architecture can be used as a common abstraction that stakeholders can use to communicate and discuss the system properties. A stakeholder is defined as an individual, team, or organization with interests in, or concerns relative to, a system. Each of the stakeholders' concerns impacts the early design decisions that the architect makes. *Architectural drivers* [26] define the concerns of the stakeholders that shape the architecture.
- *Architecture serves as a basis for the subsequent phases of the life cycle process*
Architecture defines the gross-level structure of the system to which the subsequent life cycle detailed design and code artifacts should conform. Likewise, architecture defines the boundaries of the system and provides the constraints for the implementation. Further, the software architecture is used to guide the subsequent life cycle activities and help to achieve integrity of the system.
- *Architecture supports the quality requirements*
It is important that the architecture design supports the software system quality requirements that are required by the various stakeholders. Since software architecture represents the earliest design decisions about a system, architecture can be used to define and reason about the quality attributes. In addition, software architecture can be used to analyze the risks, sensitivity points, and trade-offs.
- *Architecture serves as a means for supporting the organization*
The motivation for architecture-driven development is not just for technical reasons but also for organizational reasons. Architecture defines the gross-level structure of the system that can be used to support work allocation, budget planning, and development structure of the project.

91.2 Architecture Modeling

Every system has an architecture whether it is complex or not. If the architecture is modeled, we can control it and benefit from an architecture-driven development. Initially, software architecture was represented using arbitrary box-and-lines notations leading to ambiguous interpretations. As such, very soon, it was acknowledged to provide more formal support for architectural modeling, both visually and textually. Together with the quest for more formal approaches, a widespread awareness has established stating that architecture should be modeled based on *architecturally significant requirements* (*architecture drivers*). These requirements are systemic in nature and usually concern the overall system or the large part of the system. Modifiability and performance requirements are examples of architecturally significant requirements. This need for modeling the architecture for different stakeholders has led to the notion of *architectural views* that is a representation of a system for a particular concern. Having multiple views of the architecture helps to separate the concerns and as such support the modeling, understanding, communication, and analysis of the software architecture for different stakeholders. Architectural views conform to *viewpoints* that represent the conventions for constructing and using a view. It is now common practice to model and document different architectural views for describing the architecture according to the stakeholders' concerns [4,8,11,17,22,32,33,37]. An architectural view is a representation of a set of system elements and relations associated with them to support a particular concern. Having multiple views helps to separate the concerns and as such support the modeling, understanding, communication, and analysis of the software architecture for different stakeholders. Architectural views conform to viewpoints that represent the conventions for constructing and using a view. An *architecture framework* is defined as the coordinated set of viewpoints that are used to define the views. A more precise definition of architecture framework is given in the ISO standard [21]:

Conventions, principles, and practices for the description of architectures established within a specific domain of application and/or community of stakeholders

Different architectural frameworks have been proposed in the literature. Examples of architectural frameworks include Kruchten's 4+1 view model [22], the Siemens Four View Model [17], and the Views and Beyond (V&B) approach [8]. An example list of architecture frameworks is shown in Table 91.1.

TABLE 91.1 Example Architecture Frameworks

Architecture Framework	Viewpoints
Rational unified process/ Kruchten's 4+1 [22]	1. The logical view contains the key abstractions from the problem domain 2. The implementation view represents the system from a programmer's perspective and includes the implementation units 3. The process view documents the tasks—processes and threads 4. The deployment view documents the various physical nodes for the most typical platform configurations 5. The use case view or “plus-one view” contains use cases and scenarios of architecturally significant behavior
V&B approach [8]	Three style categories 1. Module styles—How it is structured as a set of implementation units 2. Component-and-connector (C&C) styles—How it is structured as a set of elements that have runtime behavior and interactions 3. Allocation styles—How it relates to nonsoftware structures in its environment
RM-ODP [33]	1. The enterprise viewpoint describes the business requirements 2. The information viewpoint describes the information managed by the system and the structure and content type of the supporting data 3. The computational viewpoint describes the functionality provided by the system and its functional decomposition 4. The engineering viewpoint describes the distribution of processing performed by the system to manage the information and provide the functionality 5. The technology viewpoint describes the technologies chosen to provide the processing, functionality and presentation of information

Initially, architectural frameworks were proposed with a fixed set of viewpoints. Because of the different concerns that need to be addressed for different systems, the current trend recognizes that the set of views should not be fixed but multiple viewpoints might be introduced instead. As such, recent architectural frameworks such as the V&B approach provide mechanisms to adapt existing viewpoints or to add new viewpoints. Organizing the system as a set of viewpoints has also been addressed in enterprise application system using the so-called enterprise architecture frameworks. Examples include the early Zachman Framework for Enterprise Architecture [39], The Open Group Architecture Framework (TOGAF) [38], and the ISO (ISO/IEC 10746) Reference Model of Open Distributed Processing (RM-ODP) [30].

So far, most architectural viewpoints seem to have been primarily used either to support the communication among stakeholders or at the best to provide a blueprint for the detailed design. From a historical perspective, it can be observed that viewpoints defined later are more precise and consistent than the earlier approaches but a close analysis shows that even existing viewpoints lack some precision. Moreover, since existing frameworks provide mechanisms to add new viewpoints, the risk of introducing imprecise viewpoints is high. The development of a proper and effective architecture is highly dependent on the corresponding documentation. An incomplete or imprecise viewpoint will impede the understanding and application of the viewpoints to derive the corresponding architectural views and likewise lower the quality of the architectural document.

In Figure 91.1, we show the conceptual model for architectural view modeling that is based on the ISO/IEC 42010 recommended standard for architectural description [19]. The left part of the figure shows basically the definition of the architectural drivers. A system has one or more stakeholders who have interest in the system with respect to one or more concerns. Concerns can be functional concerns or quality concerns. The right part of the figure focuses on the architectural views for the different concerns. Each system has an architecture, which is described by an architectural description. The architectural description consists of a set of views that correspond to their viewpoints. Viewpoints aim to address the stakeholder's concerns. Functional concerns will define the dominant decomposition

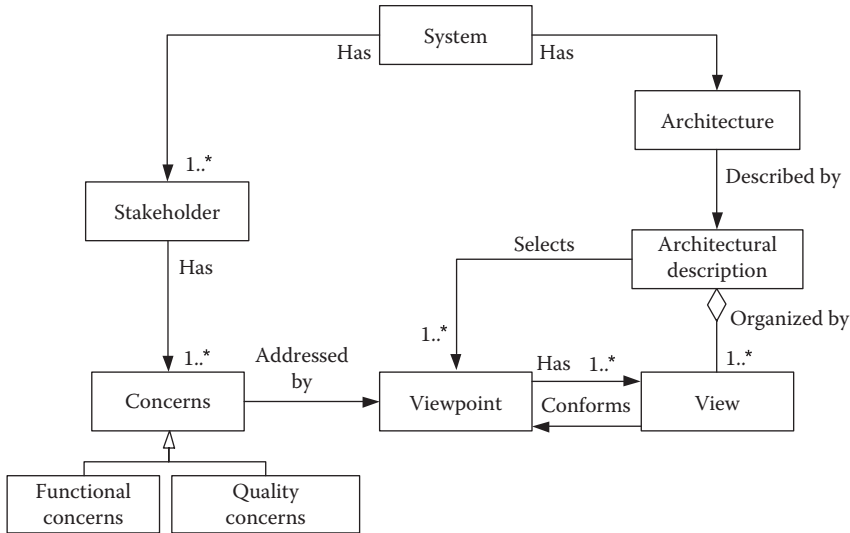


FIGURE 91.1 Conceptual model for architectural views based on ISO/IEC 42010.

along architectural units that are mainly functional in nature. On the other hand, quality concerns will define the dominant decomposition of the architecture along architectural units that explicitly represent quality concerns.

91.3 Architecture Design Methods

Various different software architecture design methods have been published in the literature. In general, the architecture design process is defined early in the software life cycle process. The main activities and a global architecture design process are given in Figure 91.2.

The process is typically defined as an iterative process in which one can iterate to previous activities if needed. In the succeeding text, we describe the activities.

91.3.1 Analyze Concerns

As stated before, architecturally significant requirements define the concerns of the stakeholders that shape the architecture. To analyze the concerns, first, the stakeholders must be identified and their concerns must be elicited. After analyzing and consolidating the elicited concerns, the concerns must be explicitly represented for guiding the subsequent architecture design activities.

91.3.2 Analyze Domain

Domain analysis can be defined as the process of identifying, capturing, and organizing domain knowledge about the problem domain with the purpose of making it reusable when creating new systems [1,9]. The UML glossary provides the following definition of the term domain:

Domain: An area of knowledge or activity characterized by a set of concepts and terminology understood by practitioners in that area.

A survey of domain analysis methods shows that these methods include the similar kind of activities. Figure 91.3 represents the common structure of domain analysis methods as it has been derived from survey studies on domain analysis methods [1,9].

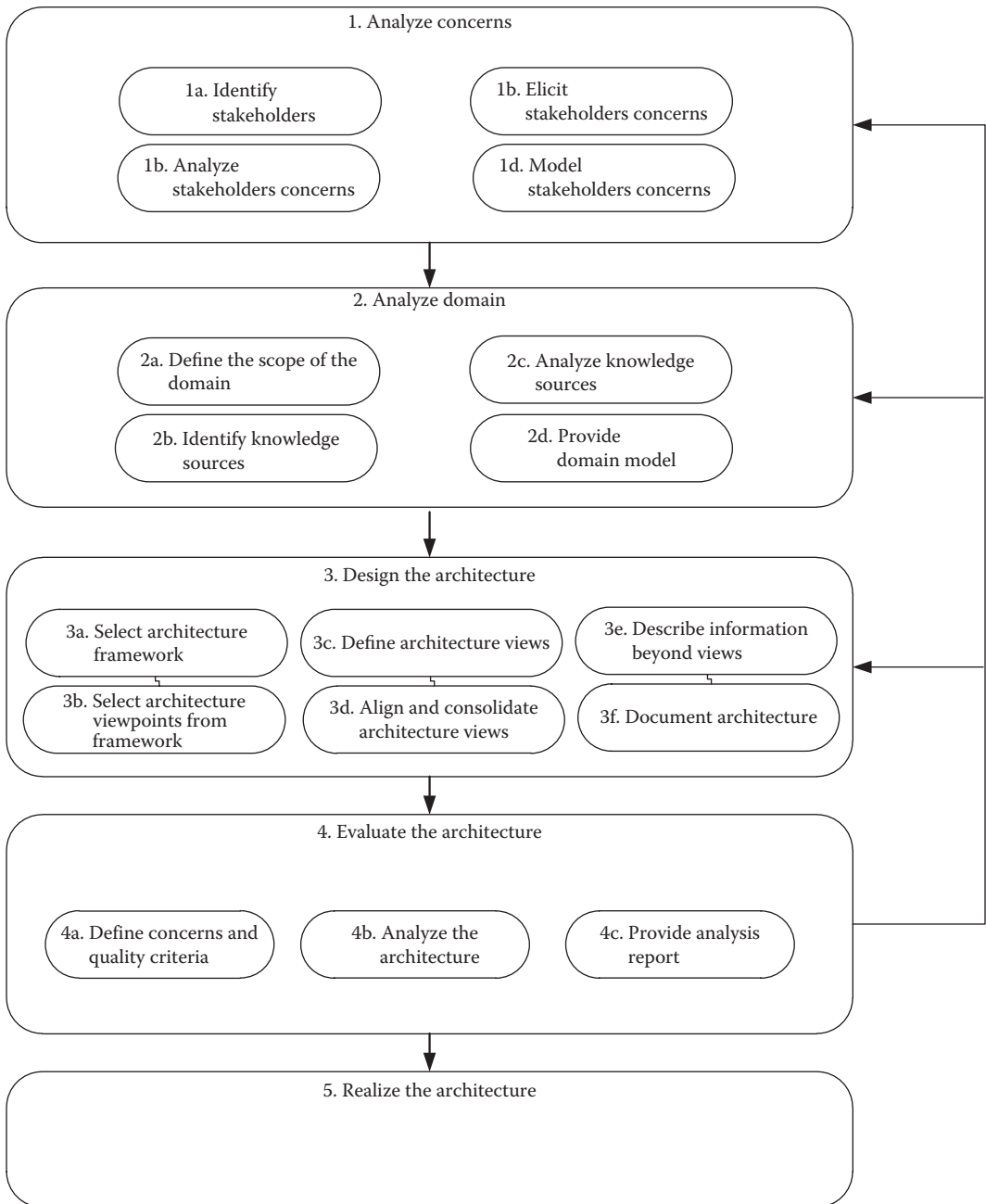


FIGURE 91.2 Example software architecture design process.

Conventional domain analysis methods consist generally of the activities *Domain Scoping* and *Domain Modeling*; *Domain Scoping* identifies the domains of interest, the stakeholders, and their goals and defines the scope of the domain. *Domain Modeling* is the activity for representing the domain, or the *domain model*. The domain model can be represented in different forms such as object-oriented language, algebraic specifications, rules, and conceptual models. Typically, a domain model is formed through a commonality and variability analysis to concepts in the domain. A *domain model* is used as a basis for engineering components intended for use in multiple applications within the domain.

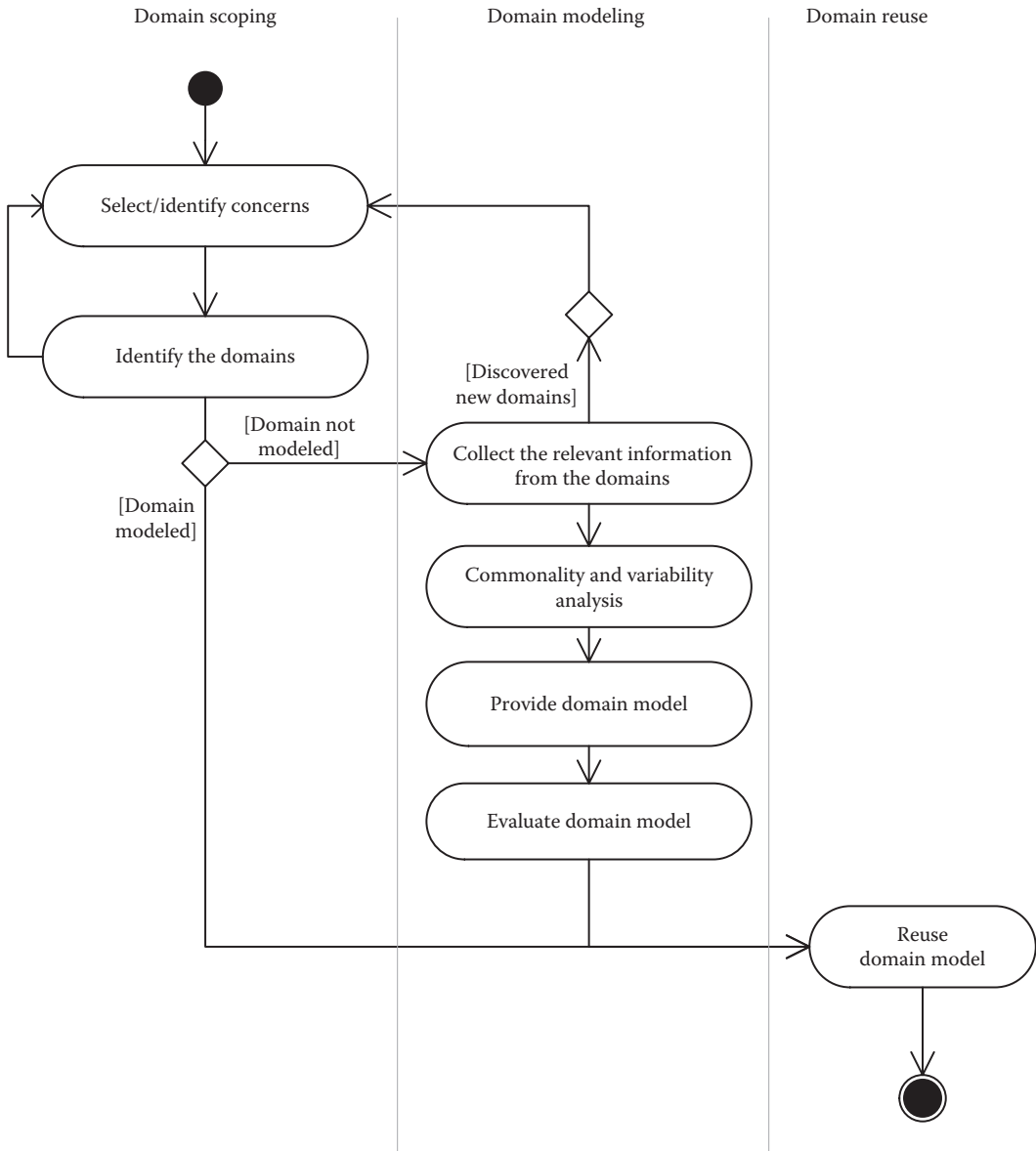


FIGURE 91.3 Common structure of domain analysis methods.

This activity results in a thorough understanding of the domain in which new concerns that were overlooked in the previous activity can be identified. For this, one can iterate back to the previous activity of concern analysis to update the identified concerns.

91.3.3 Architecture Design

Once the stakeholder concerns and the domain has been understood and modeled, the architecture design can be started. In this context, as stated before, the notion of viewpoint plays an important role in modeling and documenting architectures. Before the design process can begin, an architecture framework needs to be selected that contains the viewpoints with which the architecture views

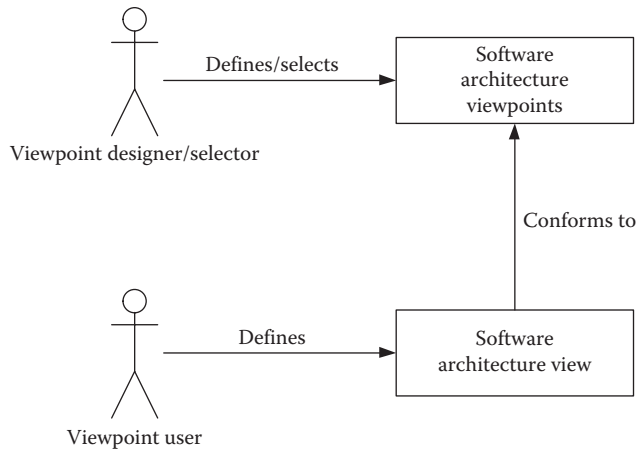


FIGURE 91.4 Viewpoint definition and view development for MPL.

will be modeled. Sometimes, new viewpoints might need to be added if the existing viewpoints are limited in modeling the stakeholder concerns. In this respect, we can distinguish between *viewpoint designer* and *viewpoint user* (Figure 91.4). The viewpoint designer either designs new viewpoints or selects viewpoints from an existing architecture framework. The viewpoint user is the software architect who uses the viewpoints to document the architecture. Viewpoint designer and viewpoint users are roles and can represent the same architect or architecture team.

After the selection of the viewpoints, the architecture views need to be aligned with each other for consistency [39]. Typically, this includes checking whether the elements in each view map to other elements of related views.

During the design, one might identify the need for new concerns or analysis of domains that were not identified before. As such, iterating to the activity *Analyze Concerns* and/or *Analyze Domain* can be necessary.

91.3.4 Architecture Evaluation

The architecture forms one of the key artifacts in the entire software development life cycle since it embodies the earliest design decisions and includes the gross-level components that directly impact the subsequent analysis, design, and implementation. The key concerns of an architecture are defined by stakeholders, an individual, team, or organization with interests in, or concerns relative to, a system. Each of the stakeholders' concerns impacts the early design decisions that the architect makes. To ensure that the concerns have been properly addressed, the architecture needs to be evaluated with respect to the defined concerns. For this, the concerns and the quality criteria are identified. Based on these, the architecture will be evaluated and an impact analysis report will be provided. The result of the evaluation process might require iteration to activities *Analyze Concerns*, *Analyze Domain*, and *Architecture Design*. We elaborate on the evaluation process in Section 91.4.

91.3.5 Architecture Realization

The final step of the architecture development cycle is the realization of the architecture.

To realize the important architecture design concerns, one of the key goals of software architecture design is to serve as a guideline for the subsequent life cycle activities. Based on the architecture design, the detailed design and subsequent activities can be carried out. Figure 91.5 shows the relationships among these concepts. An architecture description is defined using architecture views and addresses a set of concerns. The architecture is followed by the detailed design that realizes the architecture and that

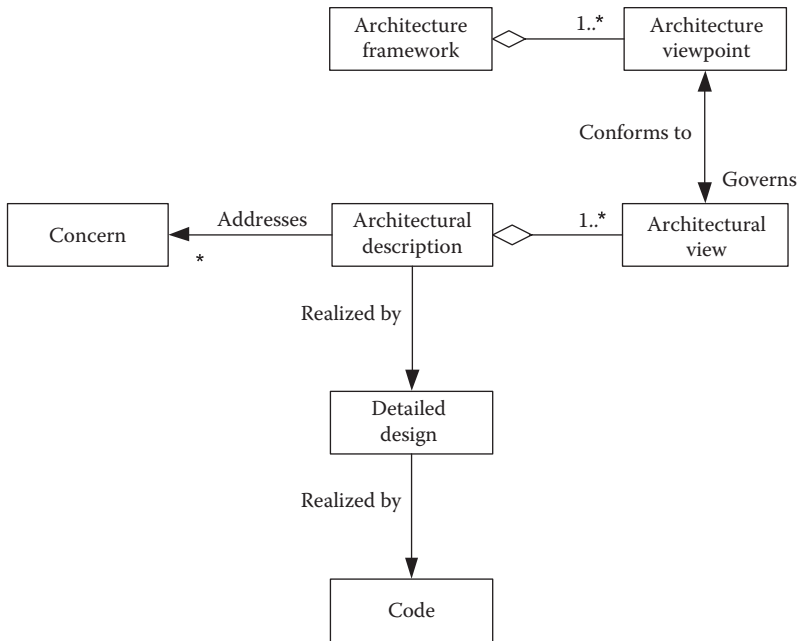


FIGURE 91.5 Relation between architecture viewpoint, architecture view, and detailed design.

is subsequently realized by code. To preserve the benefits of the architecture, it is important to keep the life cycle artifacts consistent with the architecture.

Software systems are rarely static and need to evolve over time due to bug fixes or new requirements. These required changes might lead to a situation in which the realization of the architecture (in the detailed design and code) is different from the architecture description itself. As such, the benefits of an architecture-driven software development approach will be lost. A proper documentation of the architecture is important to minimize the cost of software maintenance. Despite this, the evolution of the system and likewise the discrepancy between the architecture description and the resulting implementation will be highly probable. The discrepancy between the architecture description and the architecture realization is usually referred to as *architectural drift* or *architecture degeneration*. This architecture drift can be even introduced during the initial implementation of the architecture due to lack of knowledge about the architecture or stringent time-to-market constraints. Even if the initial implementation of the architecture is properly aligned with the architecture description, the constant need for new requirements very often leads to a situation that can lead to architecture drift.

The existence of discrepancies among the architecture description and its implementation is not a mere theoretical issue, but it may directly lead to increased maintenance time and cost, because the original design goals have been lost. In the extreme cases, a system's architecture may even deteriorate to a degree where further development is not feasible, leading to a complete system reimplementation. As such, preferably conformance checking of the architecture description and the implementation should be exercised over the architecture of a system during development and maintenance, with the aim of enforcing architecture consistency.

91.4 Architecture Evaluation

In the previous section, we have discussed the generic approach for designing software architecture. Various other architecture design approaches can be found in the literature. In this context, Hofmeister et al. [18] also provide a model of software architecture design approaches. The aim of

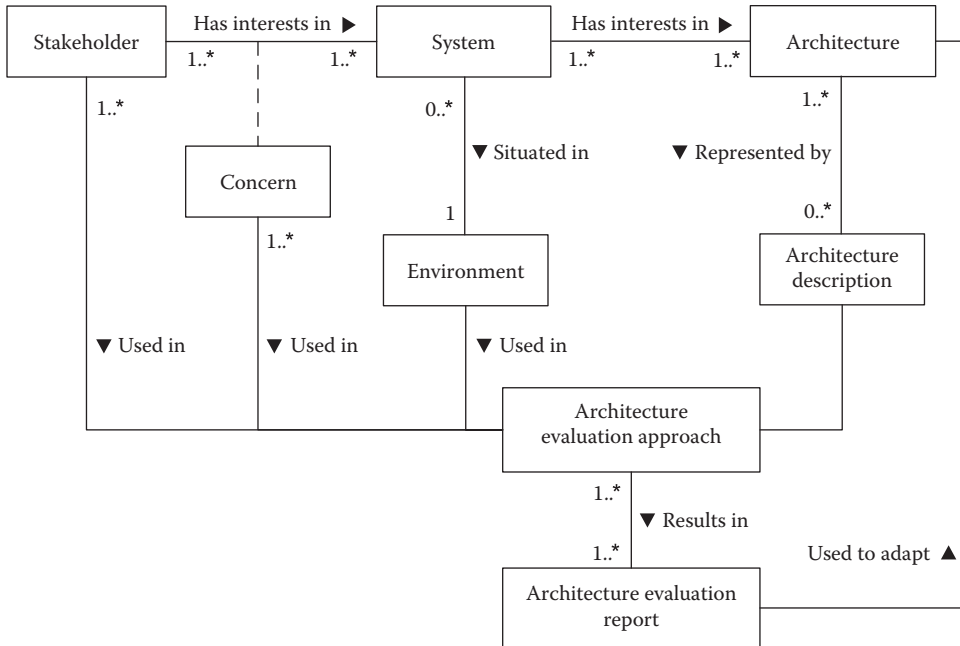


FIGURE 91.6 Conceptual model for software architecture analysis.

all these architecture design approaches is to design a software architecture that meets the stakeholder goals. Since architecture is critical for the success of a project, different architectural evaluation approaches have also been introduced to evaluate the stakeholders' concerns. A comprehensive overview and comparison of architecture analysis methods have been given by, for example, Dobrica et al. [10], and Babar et al. [2]. Kazman et al. [20] have provided a set of criteria for comparing the foundations underlying different methods, the effectiveness and usability of methods. Figure 91.6 provides a conceptual model that relates the architectural concepts with the architecture evaluation approach. Although different architecture evaluation approaches have been proposed in the literature, we can state that most of these follow the model as defined in Figure 91.6. In essence, each architecture evaluation approach takes as input the stakeholder concerns, the environment issues, and the architecture description. Based on these inputs, the evaluation results in an *architecture evaluation report*, which is used to adapt the architecture.

Figure 91.7 shows the generic conceptual process for architecture evaluation that has been based on the proposed architecture evaluation approaches. Each architecture evaluation approach appears to take as input the architecture and the quality criteria that need to be evaluated. If the architecture meets the criteria, then a decision can be made to realize the architecture, if not, then a refactoring of the architecture is required. Architecture evaluation approaches usually differ with respect to, for example, the goal of the approach, the type of inputs, the evaluation techniques, the addressed quality attributes, the stakeholders involvement, the ordering of activities, and the output results [2,20].

91.5 Research Issues

It is now generally accepted that software architecture design plays a fundamental role in coping with the inherent difficulties of the development of large-scale and complex software systems. Research on architecture design in the last two decades has resulted in different useful techniques and approaches. Different architectural modeling approaches for representing multiple views of the architecture have been proposed. Multiple architectural patterns have been introduced in literature to support the quality

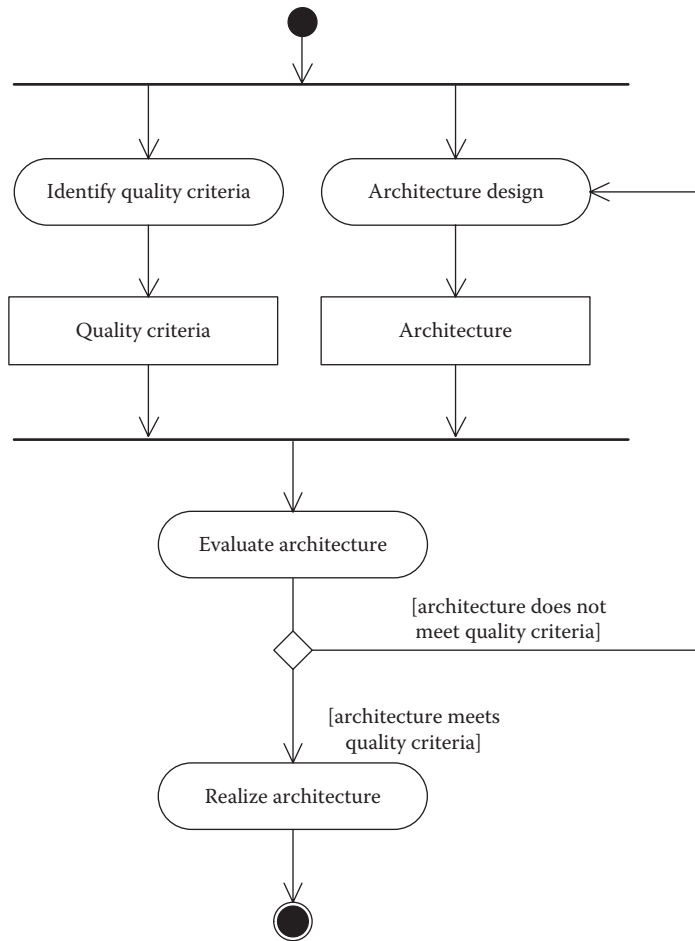


FIGURE 91.7 Generic architecture evaluation process.

of architecture [3,6,22]. A broad range of architecture analysis approaches has been proposed to analyze the architecture before it is implemented [15,34]. We can state that the software architect is now equipped with a broad set of techniques to design a software architecture that aims to meet the stakeholder concerns. Using these approaches, the software architect makes a wide range of design decisions that leads to the selection of a particular design alternative. Yet, there are indeed still challenging issues in the state of the practice and different research questions need to be tackled in the architecture design community [35]. We list some of the important research directions:

91.5.1 Modeling Quality Concerns

In the literature, initially a fixed set of viewpoints has been proposed to model and document the architecture. Because of the different concerns that need to be addressed for various systems, the current trend recognizes that the set of views should not be fixed but *open-ended*. An important example in this context is the V&B approach as proposed by Clements et al. [8]. Hereby, the authors propose the notion of architectural style that is a specialization of architectural element types and relationship types, along with any constraints. The notion of styles makes the V&B approach adaptable and open-ended since the architect is not bound to a fixed set of views but can in principle define any style that is needed. Certainly, existing architectural modeling approaches are important for representing the structure and

functionality of the system and are necessary to document the architecture systematically. An analysis of the existing multiview approaches reveals that they are primarily focused on functional concerns and appear to be incomplete when considering nonfunctional concerns [40]. The ISO/IEC 42010 standard is intentionally not specific with respect to concerns that can be addressed by views. In approaches such as the V&B approach, nonfunctional concerns appear to be implicit in the different views but no specific style has been proposed for this yet. One could argue that for addressing nonfunctional concerns, software architecture analysis approaches have been introduced. The difficulty here is that these approaches usually apply a separate quality model, such as queuing networks or process algebra, to analyze the quality properties. Although these models represent often precise calculations, they do not depict the decomposition of the architecture, and an additional translation from the evaluation of the quality model needs to be performed. To represent nonfunctional concerns more explicitly, architectural views should also be able to model the decomposition of the architecture based on the required nonfunctional concern. Viewpoints for nonfunctional concerns will typically overlap or crosscut existing viewpoints based on functional concerns. The crosscutting nature of nonfunctional concerns can be both observed *within* an architectural view and *across* architectural views. Aspect-oriented software development (AOSD) promotes the separation of crosscutting concerns to increase modularity. Hereby, crosscutting concerns are separately represented as first-class abstractions (aspects) and woven into the *base code*. To cope with the crosscutting concerns at the architecture design level, we believe that AOSD techniques are necessary [6].

Appropriate models for quality concerns can support the architecture design process to design for quality. In addition, different approaches have been defined that provide guidelines for designing for various qualities. *Architectural Perspectives* [32] are a collection of activities, tactics, and guidelines to modify a set of existing views to document and analyze quality properties. Architectural perspectives as such are basically guidelines that work on multiple views together. An important observation hereby is that the crosscutting nature of quality concerns can be both observed *within* an architectural view and *across* architectural views. Architectural tactics [3] aim at identifying architectural decisions related to a quality attribute requirement and composing these into an architecture design. Defining explicit viewpoints for quality concerns can help to model and reason about the application of architectural tactics.

91.5.2 Model-Driven Approach for Architecture Design

Architecture design is basically about *modeling* the system from different perspectives. Historically, *models* have had a long tradition in software engineering and have been widely used in software projects. The primary reason for modeling is usually defined as a means for communication, analysis, or guiding the production process. Models are different in nature and quality and different classifications of models have been provided in the literature. Mellor et al. [25] make a distinction between three kinds of models, depending on their level of precision. A model can be considered as a *sketch*, as a *blueprint*, or as an *executable*. According to [25], an executable model is a model that has everything required to produce the desired functionality of a single domain. Executable models are more precise than sketches or blueprints and can be interpreted by model compilers. A similar classification of models is defined by Fowler [15] who suggests a distinction based on three levels of models, namely, *conceptual models*, *specification models*, and *implementation models*.

In model-driven software development, the concept of *models* can be considered as executable models as defined by the aforementioned characterization of Mellor et al. [25,34]. In model-driven software development, models are not mere documentation but become “code” that are executable and that can be used to generate even more refined models or code. This is in contrast to model-based software development in which models are used as blueprints at the most.

The language in which models are expressed is defined by metamodels. As such, a model is said to be an instance of a metamodel, or a model *conforms to* a metamodel. A metamodel itself is a model that

conforms to a meta-metamodel, the language for defining metamodels [12,13,23]. Given the different levels in which the models reside in model-driven development, models are usually organized in a four-layered architecture. The top (M3) level in this model is the so-called meta-metamodel and defines the basic concepts from which specific metamodels are created at the meta (M2) level. Normal user models are regarded as residing at the M1 level, whereas real-world concepts reside at level M0.

In fact, we can state that the current architectural modeling practices can be categorized as *model-based development*, rather than *model-driven development*. In the last two to three decades, architectural modeling and the corresponding notations have evolved from simple sketches to more precise models as defined by architectural view concept. Yet, the view models can usually not be considered as executable models. Moreover, the link between architectural models and the link from architectural models are merely implicit and not formal [12].

In architecture modeling literature, the notion of metamodel is not explicitly used. Nevertheless, the concepts related to architectural description are formalized and standardized in ISO/IEC 42010:2007, a fast-track adoption by ISO of IEEE-Std 1471-2000, *Recommended Practice for Architecture Description of Software-Intensive Systems* [21]. The standard holds that an architecture description consists of a set of *views*, each of which conforms to a *viewpoint*, but it has deliberately chosen not to define a particular viewpoint. Here, the concept of view appears to be at the same level as the concept of *model* in the model-driven development approach. The concept of viewpoint, representing the language for expressing views, appears to be on the level of metamodel. Although the ISO/IEC 42010 standard does not really use the terminology of model-driven development, the concepts as described in the standard seem to align with the concepts in the metamodeling framework. In Figure 91.8, we provide a partial view of the standard that has been organized around the metamodeling framework. An *Architecture Description* is a concrete artifact that documents the *Architecture* of a *System of Interest*. The concepts *System of Interest* and *Architecture* reside at layer M0. *System of Interest* defines a system for which an *Architecture* is defined. *Architecture* is described using *Architectural Description* that resides at level M1. *Architectural Description* includes one or more *Architectural Views* that represent the system from particular stakeholder concern's perspective. Architectural views are described based on *Architectural Viewpoint*, the language for the corresponding view. *Architectural Viewpoints* are organized in *Architectural Framework*. The latter two reside at level M2. The standard does not provide a concept that we could consider at level M3, and as such we have omitted this in Figure 91.8.

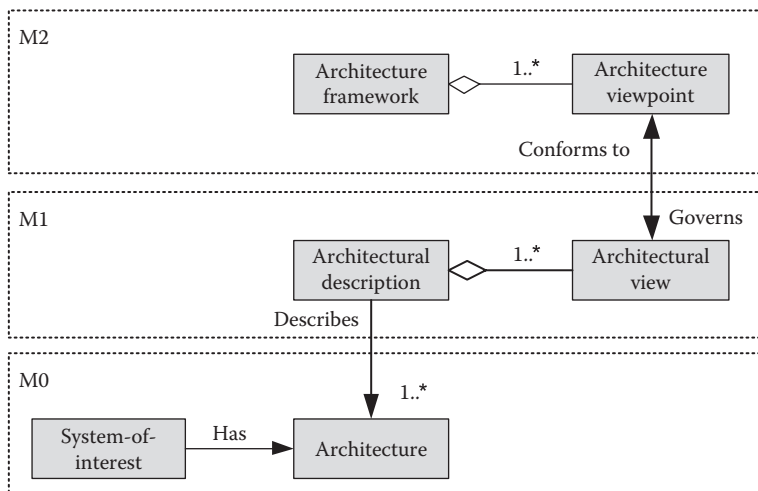


FIGURE 91.8 Architectural description concepts from a metamodeling perspective.

91.6 Summary

In this chapter, we have defined an overview of software architecture design. We have focused on three basic topics including software architecture modeling, software architecture design methods, and software architecture evaluation.

Software architecture modeling has evolved from informal box-and-line drawings to extensive architecture frameworks that include various viewpoints for modeling the architecture based on the concerns of stakeholders. Different architecture frameworks have been introduced in the literature, each with its own set of viewpoints. Earlier frameworks assumed a fixed set of viewpoints from which the architect can select the required ones to model the architecture. Recent approaches such as the V&B approach adopt an open-ended approach in which new viewpoints can be designed if deemed necessary.

The process of software architecture design usually starts in the early phases of the software development life cycle. Various architecture design methods have been introduced. We have identified five key activities that can be observed in most architecture design methods. The activities include *Analyzing Concerns*, *Analyze Domain*, *Architecture Design*, *Evaluate Architecture*, and *Architecture Realization*.

Once the architecture is designed, it can be evaluated based on quality criteria. The evaluation results in impact analysis report that might require refactoring the architecture to align it with the defined quality criteria.

Key Terms

Architectural (architecture) pattern: “An architectural pattern expresses a fundamental structural organization schema for software systems. It provides a set of predefined subsystems, specifies their responsibilities, and includes rules and guidelines for organizing the relationships between them” [5, p. 12].

Architecture description language (ADL): A language for representing a software and/or system architecture. ADLs are usually graphical languages that provide semantics that enable analysis and reasoning about architectures, often using associated tools.

Architecture framework: “Conventions and common practices for architecture description established within a specific domain or stakeholder community” (ISO/IEC 42010:2007). TOGAF and DoDAF are examples of architecture frameworks.

Architecture stakeholder: An individual, team, or organization with interests in, or concerns relative to, a system.

Concern: Any matter of interest that is held by a stakeholder.

Module: An implementation unit of software that provides a coherent set of responsibilities.

Relation: A definition of how elements cooperate to accomplish the work of the system. The description of a relation names the relations among elements and provides rules on how elements can and cannot be related.

Resource: A function, method, data stream, global variable, message end point, event trigger, or any addressable facility within an interface.

Software architecture: The fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution.

View: A representation of a set of system elements and relations among them.

Viewpoint: ISO 42010 defines a viewpoint as a work product establishing the conventions for the construction, interpretation, and use of architecture views and associated architecture models.

References

1. G. Arrango. Domain analysis methods. In *Software Reusability*, W. Schäfer, R. Prieto-Díaz, and M. Matsumoto (Eds.), Ellis Horwood, New York, pp. 17–49, 1994.
2. M.A. Babar, L. Zhu, and R. Jeffrey. A framework for classifying and comparing software architecture evaluation methods, In *Proceedings of 5th Australian Software Engineering Conference*, Melbourne, Australia, pp. 309–319, April 2004.
3. F. Bachmann, L. Bass, and M. Klein. *Deriving Architectural Tactics: A Step Toward Methodical Architectural Design*, CMU/SEI-2003-TR-004, ADA413644, Pittsburgh, PA, March 2003.
4. G. Booch, J. Rumbaugh, and I. Jacobson. *The Unified Modeling Language User Guide*, Addison-Wesley, Reading, MA, 1999.
5. F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*, Wiley, New York, 1996.
6. R. Chitchyan, A. Rashid, P. Sawyer, A. Garcia, J. Bakker, M. Pinto Alarcon, B. Tekinerdogan, S. Clarke, and A. Jackson. Survey of aspect-oriented analysis and design, AOSD Europe network of excellence, no. AOSD–Europe-ULANC-9, pp. 1–259, 2005.
7. P. Clements and L. Northrop. *Software Product Lines: Practices and Patterns*, Addison-Wesley, Boston, MA, 2002.
8. P. Clements et al. *Documenting Software Architectures: Views and Beyond*, Addison-Wesley, Boston, MA, September 2011.
9. C. Czarnecki and U. Eisenecker. *Generative Programming: Methods, Tools, and Applications*, Addison-Wesley, Boston, MA, 2000.
10. D. Rowland and T.N. Howe. *Vitruvius. Ten Books on Architecture*. Cambridge University Press, Cambridge 1999, ISBN 0-521-00292-3.
11. E. Demirli and B. Tekinerdogan. Software language engineering of architectural viewpoints, in *Proceedings of the 5th European Conference on Software Architecture (ECSA 2011)*, LNCS 6903, Essen, Germany, pp. 336–343, 2011.
12. E. Demirli and B. Tekinerdogan. SAVE: Software architecture environment for modeling views, in *Proceedings of the WICSA 2011: 9th Working IEEE/IFIP Conference on Software Architecture*, Boulder, CO, pp. 355–358, June 20–24, 2011.
13. E.W. Dijkstra. The structure of the “THE” multiprogramming system. *Communications of the ACM* 18(8), 453–457, 1968.
14. L.F. Dobrica and E. Niemela. A survey on software architecture analysis methods. *IEEE Transactions on Software Engineering* 28(7), 638–653, 2002.
15. M. Fowler, S. Scott, and G. Booch. *UML Distilled, Object Oriented Series*, Addison-Wesley, Reading, MA, p. 179, 1999.
16. D. Garlan and M. Shaw. *An Introduction to Software Architecture*, Carnegie Mellon University, Pittsburgh, PA, 1994.
17. C. Hofmeister, R. Nord, and D. Soni. *Applied Software Architecture*, Addison-Wesley, NJ, 2000.
18. C. Hofmeister, P. Kruchten, R.L. Nord, H. Obbink, A. Ran, and P. America. Generalizing a model of software architecture design from five industrial approaches. In *Proceedings of the 5th Working IEEE/IFIP Conference on Software Architecture (WICSA5)*, Pittsburgh, PA, pp. 77–86, November 6–9, 2005.
19. ISO/IEC, ISO/IEC 42010. Systems and Software Engineering—Recommended Practice for Architectural Description of Software-Intensive Systems, 2007.
20. R. Kazman, L. Bass, M. Klein, T. Lattanze, and L. Northrop. A basis for analyzing software architecture analysis methods. *Software Quality Journal* 13(4), 329–355, 2005.
21. A. Kleppe. *Software Language Engineering: Creating Domain-Specific Languages Using Metamodels*, Addison-Wesley Longman Publishing Co., Inc., Boston, MA, 2009.

22. P. Kruchten. The 4+1 view model of architecture. *IEEE Software* 12(6), 42–50, 1995.
23. A.J. Lattanze. *Architecting Software Intensive Systems: A Practitioner's Guide*, Auerbach Publications, Boca Raton, FL, 2009.
24. N. Medvidovic and R. N. Taylor. A classification and comparison framework for software architecture description languages. *IEEE Transactions on Software Engineering* 26(1), 70–93, 2000.
25. S.J. Mellor, K. Scott, A. Uhl, and D. Weise. *MDA Distilled: Principle of Model Driven Architecture*, Addison-Wesley, Reading, MA, 2004.
26. H. Obbink, P. Kruchten, W. Kozaczynski, R. Hilliard, A. Ran, H. Postema, D. Lutz, R. Kazman, W. Tracz, and E. Kahane. Report on software architecture review and assessment (SARA), Version 1.0. At <http://kruchten.com/philippe/architecture/SARAv1.pdf>. February 2002.
27. D. Parnas. On the criteria for decomposing systems into modules. *Communications of the ACM* 15(12), 1053–1058, December 1972.
28. D. Parnas. On the design and development of program families. *IEEE Transactions on Software Engineering* SE-2(1), 1–9, 1976.
29. C. Peña and J. Villalobos. An MDE approach to design enterprise architecture viewpoints, in *IEEE 12th Conference on Commerce and Enterprise Computing (CEC)*, Shanghai, China, pp. 80–87, November 10–12, 2010.
30. J. R. Romero, J. M. Troya, and A. Vallecillo. Modeling ODP computational specifications using UML. *The Computer Journal* 51, 435–450, 2008.
31. B. Roy and T. C. N. Graham. Methods for evaluating software architecture: A survey. *Computing* 545(2008-545), 82, 2008.
32. N. Rozanski and E. Woods. *Software Systems Architecture—Working with Stakeholders using Viewpoints and Perspectives*, Addison-Wesley, Upper Saddle River, NJ, 2005.
33. H. Sozer and B. Tekinerdogan. Introducing recovery style for modeling and analyzing system recovery, in *Seventh Working IEEE/IFIP Working Conference on Software Architecture*, Vancouver, BC, pp. 167–176, 2008.
34. T. Stahl and M. Voelter. *Model-Driven Software Development*, Addison-Wesley, New York, 2006.
35. B. Tekinerdogan. Exploring research directions in software architecture modeling. *International Journal of Software Architecture* 1(1), 7–9, July 2010.
36. B. Tekinerdogan, C. Hofmann, and M. Aksit. Modeling traceability of concerns for synchronizing architectural views. *Journal of Object Technology* 6(7), Special Issue: Aspect-Oriented Modeling, pp. 7–25, August 2007.
37. B. Tekinerdogan and H. Sözer. Defining architectural viewpoints for quality concerns, in *Proceedings of the 5th European Conference on Software Architecture (ECSA 2011)*, LNCS 6903, Essen, Germany, pp. 26–34, 2011.
38. TOGAF. The Open Group Architecture Framework, Version 8.1.1., 1995. <http://www.opengroup.org/architecture/togaf8-doc/arch/>
39. J.A. Zachman. A framework for information systems architecture. *IBM Systems Journal* 26(3), 276–292, 1987.